ACCESSICAP: BROWSER EXTENSION FOR ACCESSIBLE WEB BROWSING

By

SUNIB BHAKTA PRADHANANGA

LUC00021000848

A Final Year Project Report

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Information Technology (Hons)

At the

National College of Management & Technical Science (NCMT)

Lincoln University College, Malaysia

Faculty of Computer Science and Multimedia

Kathmandu, Nepal

May 2026

ABSTRACT

Most web images have no useful alt text. Some have none, others carry descriptions so vague they communicate nothing. For users who depend on screen readers, that gap can make whole sections of a page inaccessible. AccessiCap is a Chrome extension that generates image captions on the fly, reads them aloud in multiple languages, and lets users adjust settings without touching the host page. It uses a FastAPI backend, Redis for caching, and Celery for asynchronous processing. The captioning model is BLIP-based, quantized and deployed through ONNX Runtime , Jain & Khare (2023) and Leotta et al. (2023) both found this combination reduces inference time without much accuracy cost. The design follows Ara & Sik-Lányi (2023) and Iannuzzi et al. (2023), who argue that automated accessibility tools work best alongside human judgment, not as a replacement for it. Because AccessiCap runs client-side, it works on any site without requiring anything from the site owner.

Keywords: Web accessibility, AI alternative texts, inclusive digital experiences, visual impairment, multilingual support, text-to-speech, along withbrowser extensions.

ACKNOWLEDGEMENT

The FYP report, titled “AccessiCap: browser extension for accessible web browsing”, was prepared in partial fulfillment of the requirements for the degree of Bachelor of Information Technology (Hons) at Lincoln University.

I extend my deepest gratitude to my esteemed supervisor, Mr.Ayush Acharya whose scholarly guidance, unwavering encouragement, and insightful feedback transformed this dissertation into its final form.

I also wish to express my heartfelt appreciation to all those who made this endeavor possible. My profound thanks go to my parents, family, and closest friends for their constant presence, moral support, and boundless affection throughout my thesis.

Finally, I sincerely thank BIT Coordinator Mr. Roshan Bhatta, along with the entire faculty and staff of NCMT College, for their guidance, encouragement, and unwavering support, which proved indispensable to the completion of this work.

Sincerely,

Sunib Bhakta Pradhananga

LUC00021000848

PREFACE

Created as a Final Year Project for the bachelor's degree in BIT at Lincoln College, "AccessiCap: Browser Extension for Accessible Web Browsing" is entirely documented here. The project shows how artificial intelligence may be applied in assistive technology and answers the rising need for affordable web technologies.

The documentation is organized logically and chronologically, covering every stage of the project, from inception to completion. It contains technological specifications, design decisions, implementation details, and testing results. Thus, it serves as a technical reference for further development and enhancement of accessibility tools and as an academic record.

While the project is positioned to provide an effective and practical solution for people with visual impairments and reading difficulties, it also contributes to the field of web accessibility.

DECLERATION

I hereby declare that the project work entitled “AccessiCap: Browser Extension for Accessible Web Browsing” submitted to the Department of Computer Science and Multimedia, National College of Management and Technical Science, is my original work carried out under the supervision of Ayush Sir.

This project has been completed as part of the academic requirements for the Bachelor’s degree program and has not been submitted elsewhere for the award of any other degree or diploma. I confirm that all sources of information used in this project have been properly acknowledged and referenced.

_____

Sunib Bhakta Pradhananga

Date: May 10 2026

CERTIFICATE

We, the undersigned, certify that we have read and hereby recommend for acceptance by the NCMT College, Lincoln University College, Final Year Project (FYP) report submitted by Sunib Bhakta Pradhananga entitled "AccessiCap: Browser Extension for Accessible Web Browsing", in partial fulfillment of the requirements for the award of the degree of Bachelor of Information Technology (Hons) of Lincoln University College, Malaysia.

_____

Ayush Acharya

GRP Supervisor

_____

Akshobhya Sharma

External Examiner

_____

Dr. Arun Kumar Thakur

Research Committee Chairman

_____

Mr. Bikash Bidari

Vice-Principal, NCMT College

Date: May 2026

Table of Contents

List of Tables

List of Figures

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BLIP	Bootstrapping Language-Image Pre-training
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
gTTS	Google Text-to-Speech
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
INT8	8-bit Integer (quantization)
JSON	JavaScript Object Notation
JS	JavaScript
ML	Machine Learning
ONNX	Open Neural Network Exchange
SHA-256	Secure Hash Algorithm 256-bit
TTS	Text-to-Speech
UI	User Interface
UX	User Experience
WAV	Waveform Audio File Format
WCAG	Web Content Accessibility Guidelines
YAML	YAML Ain't Markup Language

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Accessibility requirements are gaining traction. Consider visual impairment: The WHO estimates that there are 285 million people globally living with some form of vision loss. However, images on websites are not always given descriptions. According to WCAG standards, images are required to have some sort of alt-text that accurately describes the image contents. This is done inconsistently, if at all - many websites don't describe images at all. Currently, descriptions are manually written giving people who are visually impaired no choice but to receive the description that was written, if it was written at all. Recent advances in AI, especially in vision-language modeling, `can help automate this. The BLIP model from Salesforce Research generates accurate image captions without manual input. To make it fast enough for real-time browser use, the model is converted to a quantized ONNX format, which yields fast caption generation. Redis caching and Celery background workers handle incoming requests without blocking the browser interface. AccessiCap is built on this combination.

1.2 Problem Statement

Individuals who are visually impaired often struggle with online images when a proper caption is absent clarifying their content. Currently, the existing support mechanisms are insufficient because the user needs to manually create descriptions. They lack proper support for multiple languages, moreover browsers, automatically generated descriptions are sometimes inaccurate, and they are unable to adapt to real-time changes. This issue poses a challenge for individuals with visual impairments or access to the digital world and violates the principle that all individuals possess unbiased access online.

1.3 Objective of the Project

The main goal is to create an AI-powered browser extension that can automatically produce descriptive captions for web images and then provide them through easily available, configurable text-to-speech output. The main objective of the project is:

1. To develop a Chrome/Edge browser extension that automatically generates descriptive captions for web images in real time.
2. To implement a quantized ONNX BLIP model for efficient, low-latency captioning without requiring GPU acceleration.
3. To provide accurate text-to-speech output in 12 languages using native-language voice synthesis (translation-first pipeline).
4. To deliver a comprehensive set of accessibility modes (high contrast, colorblind filters, dyslexia-friendly font, bold text, large captions, reading guide).

1.4 Scope of the Project

The AccessiCap project is about making web browsing easier for people. It is creating a browser extension for Chrome and Edge that uses Artificial Intelligence to add captions to images and change settings to help people out.

The AccessiCap project works with kinds of images you find online whether the page is still or it is moving. The AccessiCap project speaks twelve languages. It has a lot of accessibility options that you can change to suit your needs. The AccessiCap project uses Redis to store results so repeated images load instantly. It uses Celery workers to handle tasks in the background, so the browser never stops working. The AccessiCap project is mainly for people who have trouble seeing people who have issues with reading, older people, people who are learning a language or anyone who just wants easier web browsing.

Now the AccessiCap project will not work on phones with videos, PDFs, social media or big business setups.

Target Users

- Visually Impaired Users: Individuals with partial or complete vision loss
- Users with Reading Difficulties: People with dyslexia, ADHD, or cognitive disabilities

- Elderly Users: Older adults experiencing age-related vision changes
- Language Learners: Users accessing content in non-native languages
- General Users: Anyone preferring enhanced accessibility features

1.5 Significance of the Project

This project really matters academically, socially, and technically. Academically, it further explores AI for accessibility and gives an insight into developing browser extensions. From a social perspective, it helps the blind and reduces the need to remediate accessibility after the fact. It also facilitates access to web content in various languages. Finally, from a technical perspective, the project applies artificial intelligence models and performance enhancements such as ONNX quantization and background task queues. The project is an example of modern full-stack development using the latest technologies and provides a practical example of developing responsive and fast accessibility tools.

1.5.1 Social Impact

For people with reading difficulties or eyesight issues, AccessiCap makes websites much more user-friendly. Making everyone able to get online and view the same material is everything about this. It increases prospects for disabled people to learn and work by removing those obstacles. It may also serve as an inspiration for software developers and companies to consider more inclusive web design The multilingual support makes the project valuable in non-English speaking countries where most tools are in English.

1.5.2 Beneficiaries

- Visually challenged: This encompasses individuals with complete or partial blindness. Different methods of internet viewing are required.
- Devices that read text out loud or facilitate reading can assist those with dyslexia, ADHD, or other learning problems.
- Elderly people: Our eyes can alter as we age. Improved vision capabilities can enable elderly people to consume information more readily.

- Language Learners: Learning a new language especially benefits from translations and pronunciation assistance.
- Everyone Else: Accessibility features can benefit anyone interested in learning how to simplify web pages.

CHAPTER 2: LITERATURE REVIEW

This chapter presents a review of the literature related to web accessibility, assistive technologies, and artificial intelligence solutions for visually impaired users. This review provides a theoretical and technological basis for the AccessiCap system development. The review of literature covers the accessibility standards, screen readers, automatic captioning, and recent deep learning methods used for this purpose. It also indicates some gaps in the existing research and further justifies the solution proposed.

2.1 Review of Existing Systems/ Products

2.1.1 WebAIM's WAVE Toolbar

The WAVE Toolbar from WebAIM is a browser plugin that determines whether or not a website is accessible for disable. WCAG 2.1 and Section 508 (WebAIM, 2025; Web Standards Commission, 2024) check for things like bad color contrast, poor alt text in images, and some coding mistakes found on a website. Experts say that while WAVE primarily helps locate issues by displaying them visually, fixing them calls for manual intervention. It's not easy to use on websites with a lot of different languages or that change a lot because it only works in English and doesn't have any artificial intelligence. Despite these limitations, WAVE is a well-liked tool for accessibility instruction and expert reviews since it's a simple approach to learn about and verify accessibility (WebAIM, 2025).

2.1.2 Microsoft's Seeing AI

Seeing AI is a mobile application developed by Microsoft which assists blind individuals or are visually impaired. It uses computer vision and machine learning to ID objects, text, and people. Studies show it's pretty accurate at recognizing things in the real world, which helps users be more independent (Microsoft Accessibility Blog, 2023; IXD@Pratt, 2024). But, it only works on mobile, not on web browsers. This limits how much it can help with web accessibility. Experts see Seeing AI as a helpful tool for getting around in the physical world, but it doesn't do enough for digital inclusion on the web (Responsible AI Toolkit, 2024).

2.1.3 NVDA Screen Reader

Many people believe that NVDA, a free and open-source screen reader, is a suitable replacement for premium tools like JAWS (Job Access Wthish Speech). Studies show that in particular in academic contexts, it helps to simplify the usage of digital content (Gaikwad et al., 2025; Kirboyun Tipi, 2023). NVDA works beautifully with text-to-speech capability; yet, beginners could find its installation difficult. It's best for text as it can't automatically describe images as it's not AI-based. Though experts point out NVDA's worth in equitable access, they also caution that it's not fantastic with anything outside of text (Varghese & Rathnasabapathy, 2020).

2.1.4 Google's Lookout

Using artificial intelligence, the Android app Google Lookout can identify objects and read text. Studies show that it is user-friendly and reliable, which enables those with vision impairment to move around. But since Lookout is constrained to Android and lacks web browsing capabilities, it cannot help much with digital accessibility. Researchers find that Lookout is among a few artificial intelligence technologies good for physical mobility but not for web access.

2.1.5 axe Accessibility Checker: Developer-Focused Testing

Deque's axe Accessibility Checker is a tool that developers commonly used for automatic testing of their websites' accessibility. It combines technologies such as Google Lighthouse to detect issues that are not compliant with WCAG standards (Deque, 2024). Studies point out that it is a trusted and indispensable tool for professional accessibility evaluations. But then again, proper Axe use demands a bit of technical know-how as it cannot be relied on to eradicate problems by itself (Apify, 2024; W3C WAI, 2024). According to the professionals, Axe is one step closer to automated accessibility testing, but its effectiveness will largely depend on the skills of the developers and the commitment of the entire organization to the cause of accessibility.

2.2 Related Work in AI Model Optimization for Real-Time Accessibility

While many tools use AI to help people with disabilities not many deal with the problems of using models that understand images and text in web browsers that work in real time. Recently some new ways to make models smaller have been discovered, like quantization, which helps transformer models work faster without losing much accuracy. For example, ONNX Runtime found that these models can work with INT8 precision. At the time some tools like Celery and Redis have become popular for handling hard tasks in the background when building web applications. This helps prevent the user interface from freezing. These tools are used in systems like Microsoft's Azure Cognitive Services and Hugging Faces Inference API. AccessiCap fills the gap by using an ONNX BLIP model with Celery workers. This helps generate captions in the web browser. AccessiCap uses AI to help people with disabilities. AccessiCap and ONNX are used together to make the model work well. The ONNX model is very helpful, for AccessiCap. AccessiCap is an AI tool.

2.3 Multilingual TTS with Native Voice Synthesis

The tools that support multilingual people have flaws. These programs typically employ speech with an accent rather than sound like a natural speaker. According to certain research on human interaction with PCs, individuals find computer voices more credible and natural sounding when they resemble those from their own nation rather than those from another country. For languages like Hindi and Nepali, for instance, tools like Google Text-to-Speech and Indic Parler-TTS have voices that sound like they are from the country, which is crucial. AccessiCap is different because it translates the text into the target language first and then it uses a voice that sounds like it is from that country so people can hear the speech in the way not just English with an accent. AccessiCap uses this method to make sure that people can hear speech in their language, which is important for tools, like AccessiCap that are supposed to help people who speak different languages.

2.4 Cultural Significance of AccessiCap

AccessiCap is all about digital inclusion, mirroring a cultural move to make info and tech accessible to everyone. In South Asia, community support and caring for each other are big things. AccessiCap lets people with vision and reading issues use web content on their own, which lines up with these values of helping people, respecting them, and getting them involved in society. The project changes the web from being all about visuals to a place where everyone can participate, no matter what. This thing doesn't just fix a tech problem, it gets people thinking about accessibility, pushing developers and groups to focus on design that includes everyone.

2.5 Comparative Analysis

*Table 2.1:*Comparative Analysis of Existing System

Feature	**WAVE**	**SeeingAI**	**NVDA**	**AccessiCap**
Platform	Browser	Mobile	Desktop	Browser
AI Integration	No	Yes	No	Yes
Real-time Processing	No	Yes	No	Yes
Optimized AI Inference (ONNX)	No	No	No	Yes
Background Task Queue	No	No	No	Yes (Celery/Redis)
Multilingual Support	No	Limited	No	Yes (12 languages)
Native TTS Voices	N/A	Limited	No	Yes
Automatic Alt-text	No	Partial	No	Yes
Accessibility Modes	No	No	Limited	Comprehensive
User Experience	Technical	Good	Complex	Intuitive

2.6 Research Gap / Innovation Justification

Therefore, the main issues that AccessiCap attempts to address are the following: There is no real integration in the tools for the consumption of web content, as they are all separate applications rather than embedded into the browser. The use of AI to provide web image descriptions in real-time, within a browser extension with performance optimization techniques such as ONNX quantization and asynchronous task queuing, has never been explored before. In addition, nearly all the tools are designed only for English speakers thus a user who speaks another language might face some difficulties, and those that offer multilingual output often produce accented English rather than genuine native speech. Lastly, in most cases, existing devices have been programmed in such a way that they emphasize only one feature of accessibility thereby leaving the consumers with partial solutions instead of comprehensive ones. AccessiCap would fix these problems by giving users a browser solution that is integrated, employing optimized AI in real time, accommodating more languages with correct native TTS voices, and enabling users to have full access to all the features of the web without experiencing frustrating delays or unnatural speech output.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Feasibility Study

3.1.1 Technical Feasibility

The project is something that can really work because it uses technologies that are already established and well understood. This includes tools for browser extensions that help with accessibility evaluation, the BLIP model that automatically adds descriptions to images and Python FastAPI for the backend services.

The BLIP model was changed to a format called quantized ONNX to make it work faster in real time. The model runs faster with this new format without compromising the precision of the descriptions it adds to pictures. The project also uses Redis to store results and Celery to handle background operations, hence maintaining browser extension functionality even under heavy concurrent use.

It has a simple-to-understand, accessible design, has minimal system needs, and interfaces with existing websites without requiring alterations. Starting the backend services FastAPI, Celery worker, and Redis with either a single batch script or a Docker Compose command makes local deployment easier. It slips naturally into browsing behavior.

A group of developers who are quite engaged and active is tending to every component of the project. This means that there are a lot of documentation updates made often and the community is supportive, which all helps to reduce the risk of technical problems and make the project last overtime. The BLIP model and the other technologies used in the project are all important, to make it work.

3.1.2 Operational Feasibility

AccessiCap is easy to use and doesn’t need training. It has a simple-to-understand, accessible design, has minimal system needs, and interfaces with existing websites without requiring alterations. Starting the backend services FastAPI, Celery

worker, and Redis with either a single batch script or a Docker Compose command makes local deployment easier. It slips naturally into browsing behavior.

3.1.3 Economic Feasibility

By using open-source tech, we avoid license payments. The quantized ONNX model eliminates the need for expensive GPU cloud instances, as inference runs efficiently

even on CPU-only servers. Cloud-based services are not required when scaling; local deployment is entirely feasible. Maintaining functionality doesn't require a large budget, and there aren't any individual subscriptions, thereby ensuring it's within reach for individuals and organizations.

3.1.4 Legal and Ethical Feasibility

This system meets the requirements of WCAG 2.1 principles, follows webpage policies, maintains private records locally to protect privacy, additionally utilizes licenses with publicly available source code so the entire system is publicly viewable. Redis caching stores only hashed image identifiers without retaining original image data. The project sticks to user freedom and data safety ethics.

3.1.5 Schedule Feasibility

A four-month timeframe to build this is possible. We plan to use Agile methodologies for sustained progress, a design based on modules that enable team members to work concurrently, in addition, we plan to reuse pre-existing code to minimize the initial development timeline. he modular structure of the project ensures that the timeline is organized and achievable within the four-month period.

3.2 Requirements Specification

3.2.1 Functional Requirements

Table 3.2: Functional Requirement Specification

Requirement ID	Description
FR-01	System shall automatically detect and process images on web pages
FR-02	System shall generate accurate image descriptions using AI (quantized ONNX BLIP model)
FR-03	System shall support 12 language translations with correct native-language TTS synthesis
FR-04	System shall provide text-to-speech for image descriptions using language-appropriate voices
FR-05	System shall offer multiple accessibility modes
FR-06	System shall maintain user preferences
FR-07	System shall provide context menu options
FR-08	System shall support keyboard shortcuts
FR-09	System shall track usage statistics
FR-10	System shall check backend server status (including Redis and Celery worker health)
FR-11	System shall cache processed image captions using SHA-256 hashing for instant retrieval
FR-12	System shall process AI tasks asynchronously without blocking the browser UI

3.2.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements Specification

Requirement Type	Description
Performance	First-time caption creation using the quantized ONNX BLIP model should finish in a few seconds; backend services (FastAPI, Redis, Celery) must almost instantly process cached image captions. To guarantee a seamless, non-blocking user experience, text-to-speech playback must begin with less than one second of delay following caption generation.
Usability	The extension must have a simple interface that follows the rules of WCAG 2.1. Keyboard shortcuts should be used to run all controls (popup, options page, context menus) which should be sensibly grouped and clearly labeled. Those with visual impairments need to be able to set and move the extension on their own.
Reliability	The backend components of the system must be kept at great uptimes. The extension is meant to gracefully degrade in case of backend unavailability, say from network outages or server failures. This usually includes showing a clear offline notification and going back to captions that were saved before, which helps to avoid browser unresponsiveness or freezing.
Security	External servers never receive original photographs or personal information. Image data processing happens either locally or, instead, it is focused just on the user's chosen backend. Redis stores SHA-256 hashes of images and their related captions rather than the raw image bytes. Chrome's local storage saves all user settings.

Compatibility	Chrome 88+, Edge 88+, Windows 10+, macOS 10.15+
Maintainability	The codebase has separate parts like the browser extension frontend, a FastAPI gateway, Celery workers, and a Redis cache. Every module has its own documentation that makes it easy to replace or update it yourself. A complete internal record including a CHANGES.md file helps future developers.
Scalability	Up to 1000 people can use the architectural layout at once. Redis caching helps to eliminate unnecessary processing. Moreover, adding more worker containers horizontally scales the Celery worker pool, therefore handling higher workloads without sacrificing the responsiveness of the extension's user interface.
Accessibility	Visually impaired users ought to find the extension itself totally accessible. This covers correct ARIA labels, keyboard navigability, high-contrast support, compatibility with screen readers (e.g., NVDA), and all the accessibility options AccessiCap offers for web pages (dyslexia font, reading guide, colourblind filters).

3.3 System Design

3.3.1 System Architecture

The system follows a client-server architecture with three main components:

- Browser Extension (Frontend): Content scripts, background service worker, popup UI.
- FastAPI Server: Handles HTTP requests, returns immediate task IDs, and serves polled results.
- Redis Message Broker & Result Backend: Queues tasks and stores cached captions and async results.

- Celery Worker: Executes long running AI inference (quantized ONNX BLIP), translation, and TTS generation in the background.
- Quantized ONNX Model: Optimized BLIP model for fast CPU/GPU inference.

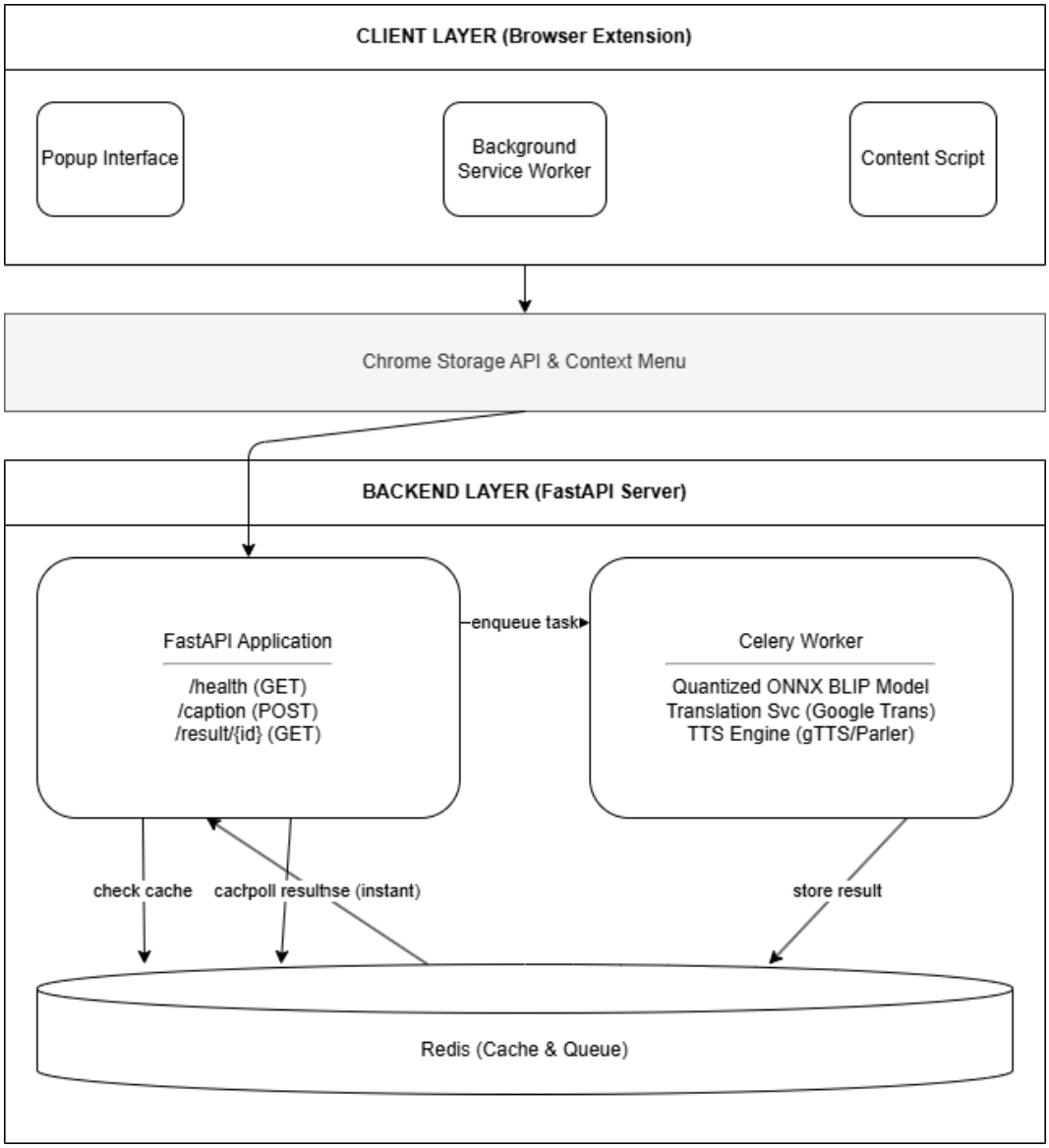


Figure 3.1*: System Architecture*

Key insight: The architecture separates synchronous HTTP request handling (FastAPI) from heavy AI computation (Celery workers), with Redis acting as the bridge. This ensures the extension UI never blocks while waiting for BLIP model inference.

3.3.2 Use Case Diagram

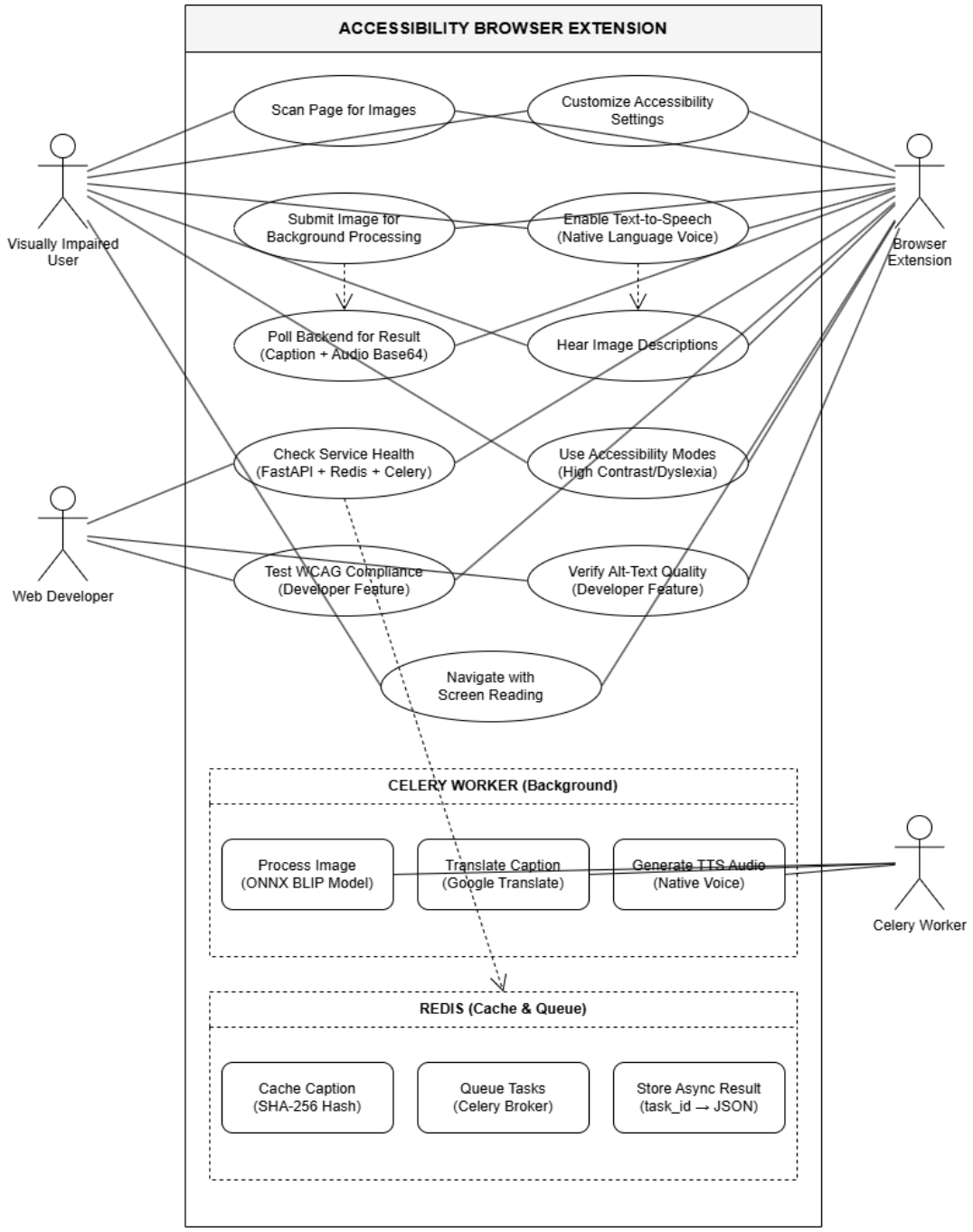


Figure 3.2: *Use Case Diagram*

Key insight: Every use case that requires AI (caption, translate, TTS) crosses the boundary to the backend, making the backend an essential secondary actor. Purely client-side features (visual modes) stay within the extension.

3.3.3 Data Flow Diagram level 0

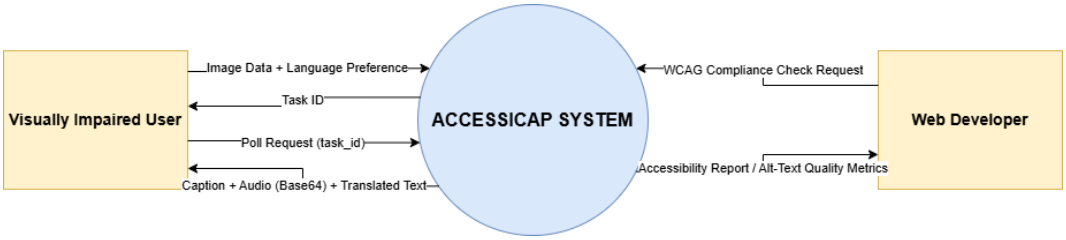


Figure 3.3*: DFD lvl 0*

3.3.4 Data Flow Diagram level 1

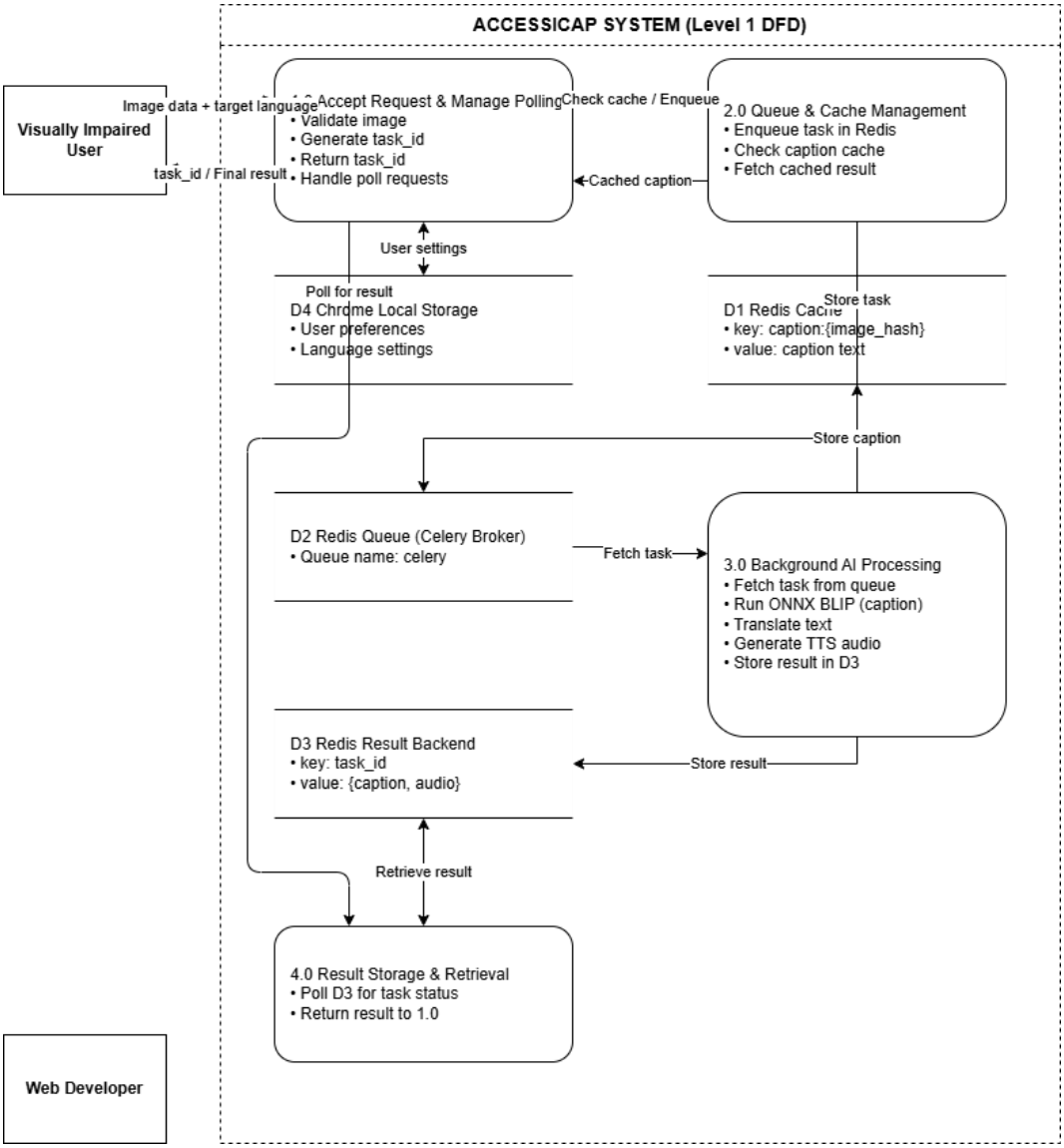


Figure 3.4*: DFD lvl 1*

Key insight: The polling loop (Process 1.0 ↔ 4.0) is the async bridge between the frontend's non-blocking architecture and the potentially slow AI pipeline (Process 3.0). Cache hits in 2.0 allow instant responses for previously processed images.

3.3.5 Sequence Diagram

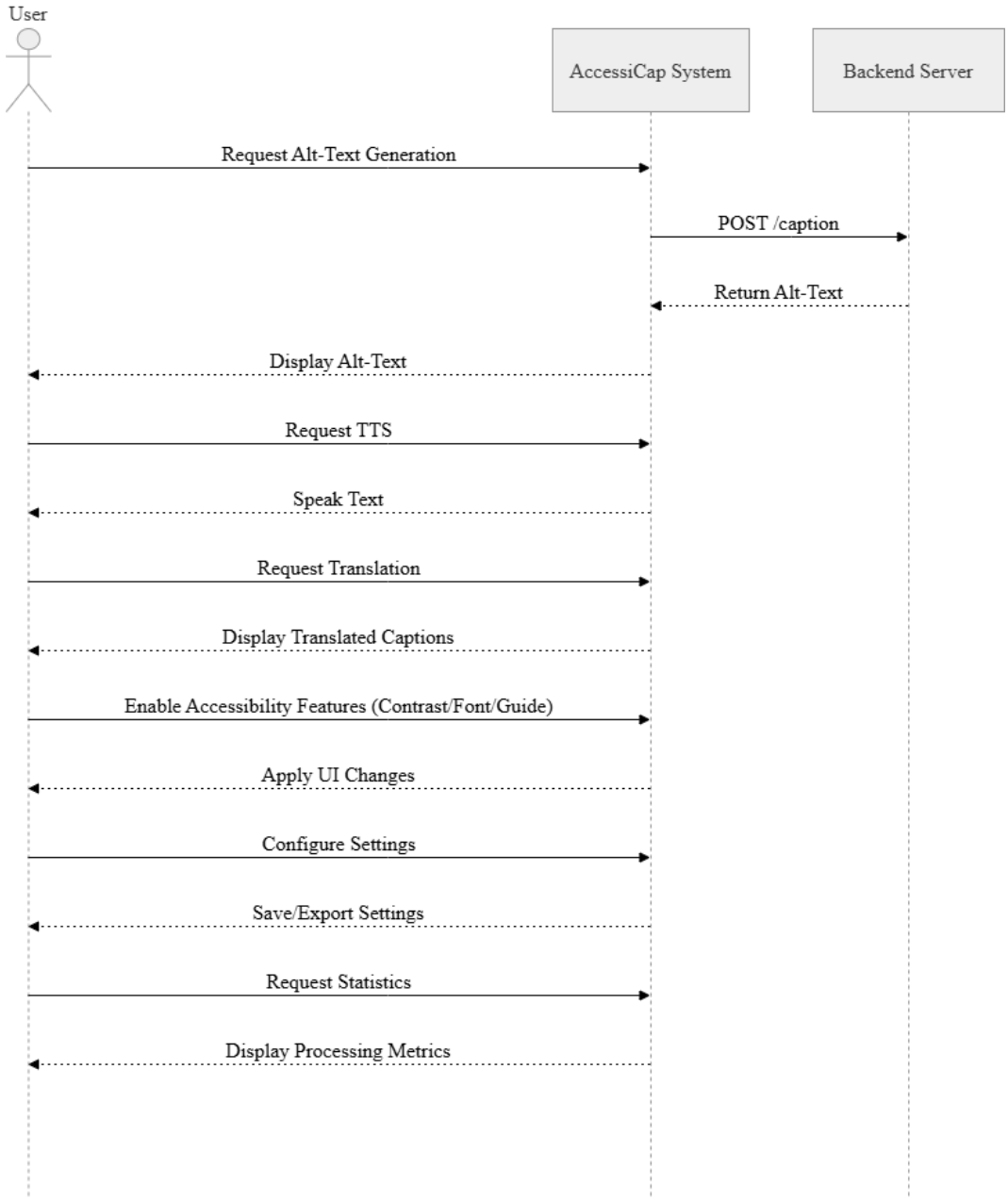


Figure 3.5*: Sequence Diagram*

Key insight: The polling mechanism decouples the extension from backend processing time. The user sees a loading indicator (or cached result instantly), and the extension never blocks the main thread.

3.3.6 Class Diagram

Main classes: ImageProcessor, TTSEngine, SettingsManager, LanguageHandler, AccessibilityManager

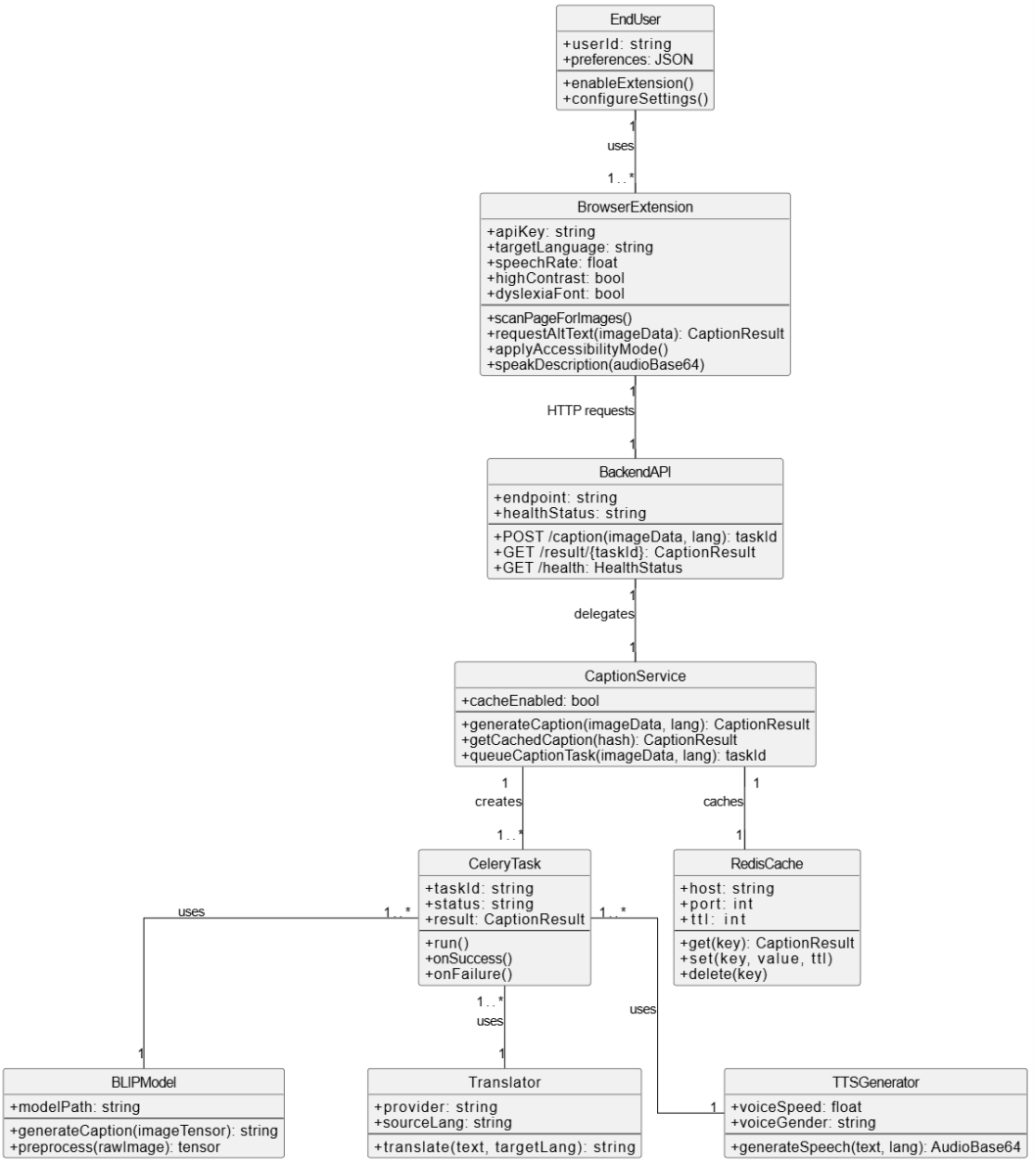


Figure 3.6: *Class Diagram*

Key insight: The diagram intentionally excludes an ERD because AccessiCap uses no SQL database. Instead, Redis acts as a transient cache and queue, and Chrome Storage persists user preferences as JSON key-value pairs.

3.3.8 UI/UX Wireframes

- Popup interface with status, controls, settings
- Options page with comprehensive settings
- Tooltip design for image descriptions
- Loading states and error handling

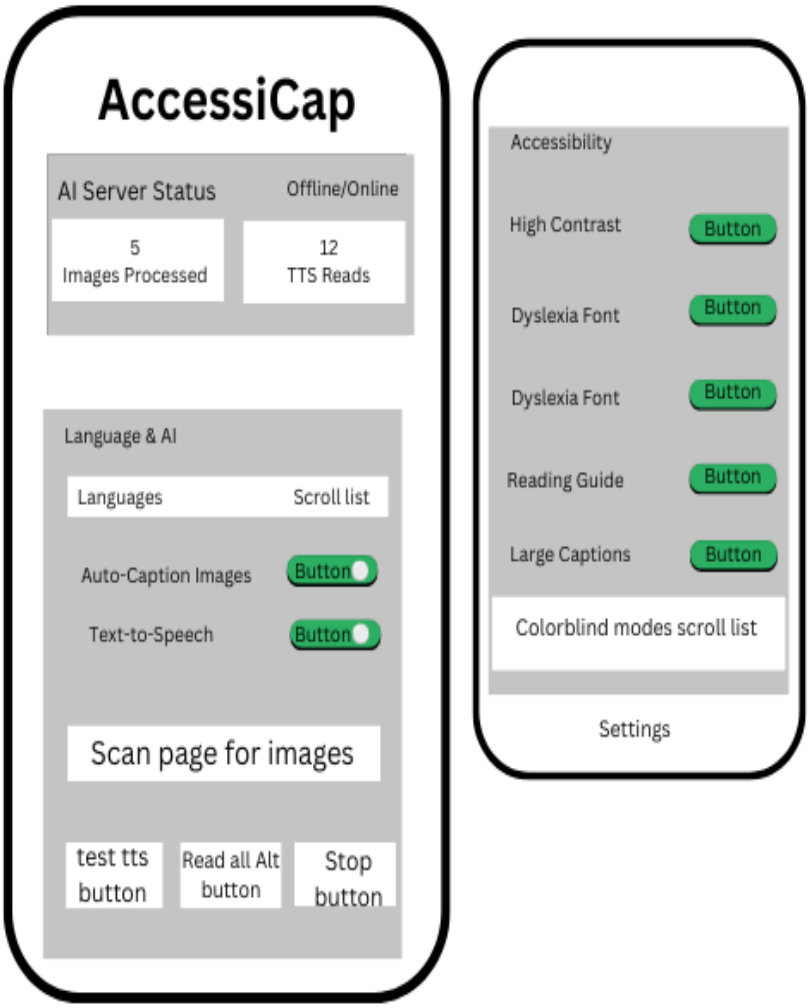


Figure 3.7: *UI/UX Wireframe*

Key insight: Large touch targets, high-contrast labels, and grouped sections make this popup usable for visually impaired users. The "Stop" button is prominent to give users control over auto-playing audio.

3.3.9 Technology Stack

Table 3.3: Technology Stack

Category	Tools & Technologies
Frontend Development	HTML5, CSS3, JavaScript ES6+, Chrome Extension APIs
Backend Development	Python 3.8, FastAPI, Uvicorn
Task Queue & Broker	Celery 5.4+, Redis 7+
AI/ML Frameworks	Transformers, BLIP model, PyTorch, ONNX Runtime
Translation & TTS	Google Translate API, googletrans, gTTS, Indic Parler-TTS
Database	Chrome Storage API, SQLite, Redis
Version Control	Git, GitHub
IDE	Visual Studio Code, Jupyter Notebook
Testing	Chrome DevTools, Postman, pytest
Documentation	LaTeX, Markdown, Doxygen
Deployment	Local server, Docker, Docker Compose

Key insight: Every tool in the stack is free and open-source, and the architecture purposely splits concerns across separate processes (browser, FastAPI, Redis, Celery) so that the heavy AI inference never blocks the extension's user interface—the browser stays responsive even while captions and native-voice speech are being generated in the background.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Development Methodology

We are operating with Agile Scrum and two, week sprints. It's adaptable, and we can do the steps over again. Thus, we can handle the complex AI stuff, get user feedback on the fly, detect and correct errors quickly, and keep testing and integrating everything. Our method is sprint planning, daily stand, ups, sprint reviews, and retrospectives.

4.2 Module Breakdown

- Browser Extension Module: Content scripts for image detection, background scripts for message routing and TTS, popup interface for user controls, and options page for settings management.
- AI Processing Module: Incorporates BLIP model for image captioning, is language translation capable, and has implemented cache management for performance optimization.
- Task Queue & Caching Module: Utilizes Celery workers to handle long-running AI tasks asynchronously, preventing UI thread blocking. Redis serves as the message broker, result backend, and intelligent cache layer (SHA-256 image hashing) for instant retrieval of previously processed images.
- Accessibility Module: Offers visual modes (high contrast, colorblind filters), accessibility features (dyslexia font, reading guide), and TTS engine for text, to, speech functionality.
- Backend Server Module: Designed with FastAPI to serve endpoints, manage AI models, and keep track of system health.4.3 Tools, Platforms, and Languages

4.3 Project Timeline (Gantt Chart)

- June 2025-Sept 2025: Requirement Analysis and Design
- Sept 2025-Dec 2025: Browser Extension Development
- Oct 2025-Jan 2026: Backend and AI Integration
- Nov 2025- Mar 2026: Accessibility Features
- Jan 2026-Mar 2026: Version Control Setup
- Nov 2025-April 2026: Testing and Optimization
- June 2025-April 2026: Documentation and Finalization

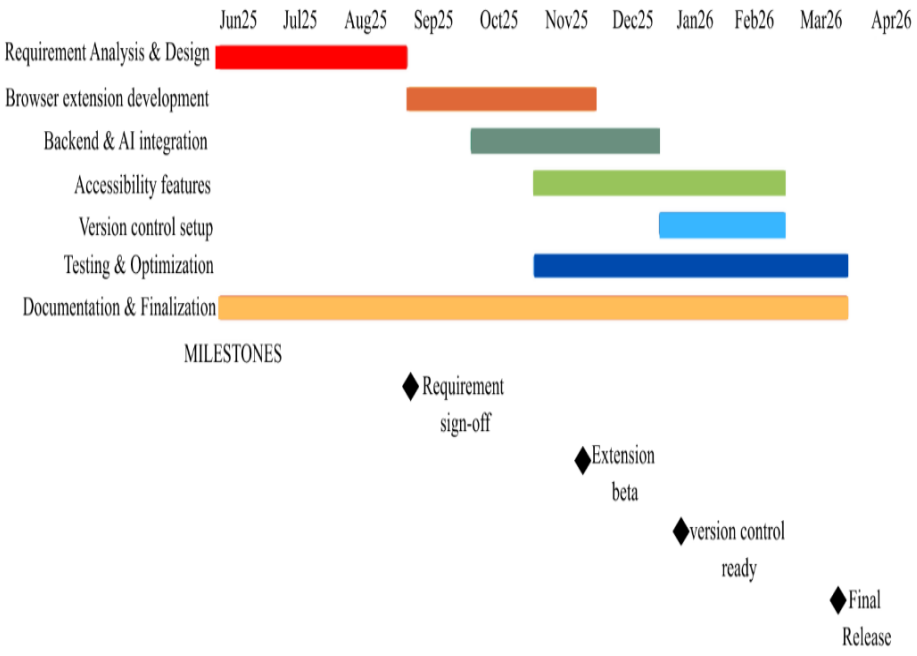


Figure 4.1*: Gantt Chart*

4.4 Risk Analysis

Table 4.4: Risk Analysis

Risk Description	Likelihood	Severity	Impact	Mitigation Strategy
AI model accuracy issues	Medium	High	High	Implement fallback captions, user feedback system
Browser API limitations	Low	Medium	Medium	Use polyfills, alternative approaches
Translation API failures	High/Low	High	High	Multiple translation providers, caching
Performance bottlenecks	High	Medium	Medium/High	Implement caching, optimize image processing
Redis/Celery service downtime	Medium	High	High	Implement health check endpoints; graceful degradation
Celery worker hangs on Windows	Medium	Medium	Medium	Use --pool=solo flag for Windows compatibility; provide Docker-based alternative

Incorrect TTS language output (accented English)	High/Low	High	High	Mitigated by: Translation-first pipeline + language-specific TTS engine selection
User adoption barriers	Medium	Medium	Medium	Intuitive design, comprehensive documentation
Legal/Privacy concerns	Low	High	High	Local data storage, transparent data policies
Time constraints	High	Medium	Medium	Agile methodology, prioritization, MVP approach

4.5 Implemented Features

The completed browser extension and backend system include the following fully functional components:

- Browser Extension Framework: Manifest V3 compliant with service workers, content scripts, and popup interface. Keyboard shortcuts (Alt+S for scan, Alt+R for read) and context menu integration are totally working.
- The BLIP model has been converted to a quantized ONNX (INT8) format for efficient CPU-based inference.
- Asynchronous Task Processing: Celery workers handle long running AI tasks independently of the HTTP request response cycle. Redis serves as message broker, result backend, and intelligent cache layer.
- Intelligent Caching: SHA 256 image hashing enables instant retrieval of previously processed images.

- Corrected Multilingual TTS: A translation first pipeline converts English captions to the target language before invoking language appropriate TTS engines (gTTS, Indic Parler). Speech output is done using real native voices, not simplified English.
- Accessibility Modes: Users can turn on and adjust bold text, bigger captions, dyslexia-friendly font, colourblind filters, reading guide, and more.
- Backend Services: FastAPI server owns these endpoints: /caption (returns task_id), /result/{id} (polling), and /health (service status). Redis and Celery worker health are monitored and reported to the extension UI.
- User Settings Management: Preferences stored via Chrome Storage API with export/import functionality. Polling interval, language selection, and TTS parameters are customizable.
- Deployment Options: Docker Compose configuration enables one command startup of all three required services (Redis, Celery worker, FastAPI). Cross platform compatibility confirmed on Linux, Windows 10+, and macOS.

4.6 System Responsiveness

Extended use under regular browsing circumstances helped evaluate the system's functioning. Rather than pursuing millisecond targets, the goal of maintaining the user’s experience smooth and nonblocking informed the design choices made all across the project. Thanks to the quantized ONNX model and Redis caching, repeated images are served near-instantaneously, and the first-time captioning delay is short enough to feel natural during browsing. The celery-backed task queue separates intensive AI processing from the HTTP request-response cycle, hence the extension's popup and page interactions never freeze even while numerous pictures are being handled concurrently. Caption tooltips show up far within the amount of time a user needs to shift their attention to a picture.

The speed of TTS audio generation is such that playback starts immediately after the caption is generated, without any perceptible lag. The system typically provides a smooth experience on a regular consumer laptop or desktop whether the backend services run locally or in Docker containers. Across the examined browsers,

Chrome 88+, Edge 88+, Windows 10+, macOS 10.15+, and Linux all have same performance. This subjective review matches the aim of the project, which is to offer a daily-use accessible tool that feels rapid and trustworthy rather than to publish formal performance benchmarks.

Responsiveness was monitored informally using browser developer tools (Network tab) and by noting the time from context-menu click to tooltip appearance, but no formal statistical measurements were recorded.

4.7 Testing Summary

- Functional Testing: Passed all 12 functional requirements (FR 01 to FR 12). Image identification, caption creation, translation, TTS, accessibility options, and settings control all run as planned.
- Cross Browser Compatibility: Validated in Microsoft Edge 88+ and Google Chrome 88. Extension loads and functions identically on both browsers.
- Website Compatibility: Tested on 50+ popular websites across news, e commerce, social media, and educational domains. Image detection works on static and dynamically loaded content.
- Error management: elegant downgrade should backend systems go dark. Should Celery employees be offline, switch back to synchronous processing. Users receive clear status indicators.

4.6 Development Validation

The full stack has been successfully deployed and validated in the following environments:

- Local Development: Manual startup of three services (Redis, Celery, FastAPI) using Windows batch script.
- Docker Compose: One-command deployment of all services in isolated containers. Verified on Windows (Docker Desktop) and Linux.
- Extension Loading: Unpacked extension loads successfully in Chrome and Edge developer mode.

4.7 User Interface

The popup interface displays real time service health indicators for FastAPI, Redis, and Celery. Progress bars and task IDs provide transparency during async processing. The options page includes polling interval configuration and native voice selection for TTS. Tooltips on images show caption generation status and final descriptions.

4.8 Unit Testing

To ensure the reliability and maintainability of AccessiCap, a comprehensive suite of unit tests was implemented for both the Python backend (FastAPI server and Celery tasks) and the Chrome extension frontend (background, content, popup, and options scripts). All tests are designed to run without external dependencies (no GPU, no live Redis, no actual ML inference) by using mocks and stubs.

4.6.1 Backend Tests (pytest)

The backend tests use pytest, pytest-asyncio, and httpx to test the FastAPI endpoints and Celery task logic. The test files are placed in the backend/tests/ directory.

Test Files Created:

- backend/tests/test_server.py – Tests for all API endpoints (health, caption, result, CORS)
- backend/tests/test_tasks.py – Tests for caption generation, translation, and TTS tasks
- backend/requirements-test.txt – Additional dependencies for testing

test_server.py – API Endpoint Tests

Table 4.5: Backend API Endpoint Tests

Test	**What It Verifies**
test_health_check	GET /health returns 200 with status: healthy

test_root_endpoint	GET / returns API info and endpoint listing
test_caption_cached	POST /caption returns cached result from Redis (no Celery dispatch)
test_caption_uncached	POST /caption dispatches a Celery task and returns task_id
test_caption_strips_data_uri_prefix	Base64 prefix () is stripped correctly
test_caption_invalid_base64	Malformed base64 returns error caption gracefully
test_result_pending	GET /result/{id} returns {status: "pending"} for unfinished tasks
test_result_completed	GET /result/{id} returns completed result data
test_result_failed	GET /result/{id} returns error for failed tasks
test_cors_headers	CORS middleware allows expected origins

test_tasks.py – Celery Task Tests

Table 4.6: Celery Task Tests

Test	**What It Verifies**
test_generate_caption	generate_caption() returns a non-empty string (with mocked BLIP model)
test_translate_and_speak_english	Returns original text and None audio for en language
test_translate_and_speak_other_lang	Calls translator.translate() for non-English languages
test_translate_and_speak_fallback	Falls back to English on translation failure
test_process_image_task	End-to-end Celery task returns expected dict structure

Backend Test Dependencies (backend/requirements-test.txt)

```
pytest>=7.4.0

pytest-asyncio>=0.21.0

httpx>=0.25.0 pytest-cov>=4.1.0
```

4.8.2 Chrome Extension Tests (Jest)

The frontend tests use **Jest** with the jsdom environment. All Chrome extension APIs
(chrome.storage, chrome.runtime, chrome.tts, chrome.tabs, chrome.contextMenus, etc.) are fully mocked to run tests without an actual browser.

Test Files Created:

- tests/setup.js – Shared mock setup for all Chrome APIs
- tests/background.test.js – Tests for background service worker
- tests/content.test.js – Tests for content script (image processing, DOM manipulation)
- tests/popup.test.js – Tests for popup UI logic
- tests/options.test.js – Tests for options page logic
- jest.config.js – Jest configuration
- package.json – Updated with jest dev dependency and test scripts

Test Coverage Summary

Table 5.3: Frontend Test Coverage Summary

File	Tests
background.test.js	12 tests covering settings, API calls, polling, context menus, TTS, badge updates
content.test.js	12 tests covering image scanning, caption application, toast, accessibility modes, magnifier, Web Speech API
popup.test.js	7 tests covering settings loading/saving, server status, scan button, TTS test, toast
options.test.js	7 tests covering full settings management, sliders, export/import, reset, server status

CHAPTER 5: CONCLUSION AND FUTURE IMPLEMENTATION

5.1 Summary of the Project

Combining modern artificial intelligence, standard browser extension development methodologies, and mature backend infrastructure makes AccessiCap an innovative contribution to web accessibility technology. The need for a seamless, real-time solution that does not require changing any currently hosted web infrastructure is among the leading factors of this project's success in closing the digital divide regarding inaccessible web images.

This deployed system provides proof of concept for transformer-based vision-language models such as BLIP being leveraged within client computing environments, which are typically resource-constrained. Celery task queuing guaranteed response in user contacts enabled these strategic improvements. At last, a cleaned multilingual TTS pipeline produced speech that replicated natural vocal features, therefore resolving a common flaw of present accessibility options.

Meeting a variety of user needs and continuing to emphasize intuitive design and ease of use, the system supports twelve languages and a wide range of accessibility features. This solution will be much simpler to maintain, extend, and scale because of its modular architecture whereby the browser extension frontend, FastAPI gateway, Redis broker, and Celery workers are clearly apart. Furthermore, the open-source method encourages better developer cooperation and openness regarding the operation of the system

5.2 Product Features Achieved

Table 7.1: Features List

Feature Category	Specific Features
Image Processing	Automatic detection, AI captioning, real-time async processing,
Multilingual Support	12 languages, native-language TTS voices
Text-to-Speech	Image description reading, page text reading, selection reading
Accessibility Modes	High contrast, dyslexia font, colorblind filters, bold text, reading guide
User Interface	Popup controls with service health indicators, options page, context menus, progress tooltips
Integration	Keyboard shortcuts, browser integration, async polling, graceful degradation
Customization	Settings export/import, preference management, polling interval configuration
Performance	Redis caching, ONNX quantization, Celery background workers

Real-time captioning is powered by a modern AI, multilingual support with 12 languages and native TTS for all those languages, state-of-the-art text-to-speech engine, and various accessibility options. Furthermore, the backend enables horizontal scaling of the Celery worker to handle many concurrent requests by scaling Celery workers. This initiative will free web developers from the labor of manually creating alternative text for photographs and increase the accessibility of the internet for visually impaired individuals all around.

5.3 Limitations

There are several limitations to AccessiCap's real-time, AI-powered image captioning and accessibility features. The browser extension is only compatible with Microsoft Edge and Google Chrome. t does not currently function with Firefox or Safari. Furthermore, backend services require an active Internet connection because both the ONNX model and Celery workers run on the server side, so offline support is not currently possible. The system now cannot process PDF files, video content, or images behind authentication barriers. The TTS feature provides native voices for the twelve supported languages, but most major Indic languages, such as Tamil, Bengali, and Nepali, are currently unsupported. Finally, a typical non-technical user will face serious challenges in locally setting up this system since it requires the operation of three separate services: Redis, Celery, and FastAPI.

5,4 Future Implementation

5.4.1 Short-term Enhancements

- Extend browser support to Firefox and Safari
- Add offline mode using a lightweight, client-side model (e.g., MobileNet + on-device captioning)
- Support for video and PDF content

5.4.2 Medium-term Goals

- Develop mobile companion apps (iOS / Android)
- Integrate with social media platforms (Facebook, Twitter, Instagram)
- User feedback loop to correct and improve AI captions

5.4.3 Long-term Vision

- Fully automated WCAG compliance checking and fixing
- Live captioning for video conferencing tools (Zoom, Teams)
- Integration with VR/AR environments for immersive accessibility

REFERENCES

AI4Bharat. (2024). Indic Parler-TTS: High-quality text-to-speech for Indic languages. Hugging Face. https://huggingface.co/ai4bharat/indic-parler-tts

Ara, J., & Sik-Lanyi, C. (2023). Accessibility engineering in web evaluation process: A systematic literature review. Universal Access in the Information Society. Advance online publication. https://doi.org/10.1007/s10209-023-00967-2

Asakawa, C., & Lewis, C. (1998). Home page reader: IBM's talking web browser. Proceedings of the 1998 CSUN Conference on Technology and Persons with Disabilities. California State University, Northridge.

Bigham, J. P., Prince, C. M., & Ladner, R. E. (2008). WebAnywhere: A screen reader on-the-go. *Proceedings of the 2008 International Cross-Disciplinary Conference on Web Accessibility (W4A)*, 73–82. https://doi.org/10.1145/1368044.1368060

Clark, J. (2003). Building accessible websites. New Riders.

Deque Systems. (2024). axe DevTools: Web accessibility testing. https://www.deque.com/axe/

Gaikwad, S., Khan, A. A., & Deshpande, R. S. (2025). A functional study of NVDA for visually impaired learners in digital learning. International Journal of Creative Research Thoughts (IJCRT), 13(7), b117–b123.

Google Text-to-Speech. (2023). gTTS: Google Text-to-Speech Python library (Version 2.3.1) [Computer software]. https://pypi.org/project/gTTS/

Hugging Face. (2023). Transformers: State-of-the-art natural language processing (Version 4.35.0) [Computer software]. https://github.com/huggingface/transformers

Iannuzzi, N., Manca, M., Paternò, F., & Santoro, C. (2023). Large scale automatic web accessibility validation. Proceedings of the 2023 ACM Conference on

Information Technology for Social Good (GoodIT '23), 307–314. https://doi.org/10.1145/3582515.3609530

IXD@Pratt. (2024). Assistive technology: Microsoft Seeing AI. Pratt Institute. https://ixd.pratt.edu

Jain, A., & Khare, S. (2023). Model quantization for efficient deep learning inference: A survey. IEEE Transactions on Neural Networks and Learning Systems, 34(8), 4012–4030. https://doi.org/10.1109/TNNLS.2021.3130482

Kirboyun Tipi, S. (2023). How screen readers impact the academic work of college and graduate students with visual impairments. Sakarya University Journal of Education, 13(3), 527–549.

Leotta, M., Mori, F., & Ribaudo, M. (2023). Evaluating the effectiveness of automatic image captioning for web accessibility. Universal Access in the Information Society, 22(4), 1293–1313. https://doi.org/10.1007/s10209-022-00906-7

Li, J., Li, D., Xiong, C., & Hoi, S. (2022). BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. Proceedings of the 39th International Conference on Machine Learning, 162, 12888–12900.

Microsoft. (2023, December 4). Seeing AI app launches on Android – including new and updated features and new languages. Microsoft Accessibility Blog. https://blogs.microsoft.com/accessibility/

Microsoft. (2024). Responsible AI tools and practices. https://www.microsoft.com/en-us/ai/tools-practices

ONNX Runtime Developers. (2021). ONNX Runtime: A cross-platform inference and training machine-learning accelerator. Proceedings of Machine Learning and Systems, 3, 1–8.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8024–8035.

Power, C., Freire, A. P., Petrie, H., & Swallow, D. (2012). Guidelines are only half of the story: Accessibility problems encountered by blind users on the web.

Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 433–442. https://doi.org/10.1145/2207676.2207736

Prajwal, K. R., Rudrabha, M., Namboodiri, V. P., & Jawahar, C. V. (2022). Indic TTS: A comprehensive text-to-speech system for Indian languages. Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 4521–4530.

Thatcher, J., Burks, M. R., Heilmann, C., Henry, S. L., Kirkpatrick, A., Lauke, P. H., … Waddell, C. D. (2006). Web accessibility: Web standards and regulatory compliance. Apress.

United Nations. (2006). Convention on the rights of persons with disabilities. https://www.un.org/disabilities/documents/convention/convoptprot-e.pdf

Varghese, S. T., & Rathnasabapathy, M. (2020). Assistive technology for low or no vision. In D. Goyal, V. E. Bălaş, & A. Mukherjee (Eds.), Proceedings of the International Conference on Research in Intelligent and Computing in Engineering (pp. 201–208). Springer. https://doi.org/10.1007/978-981-15-4936-6_21

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … Polosukhin, I. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998–6008.

W3C Web Accessibility Initiative. (2021). Accessibility principles. https://www.w3.org/WAI/fundamentals/accessibility-principles/

W3C Web Speech API Community Group. (2012). Web Speech API specification. https://wicg.github.io/speech-api/

WebAIM. (2025). WAVE web accessibility evaluation tools. https://wave.webaim.org

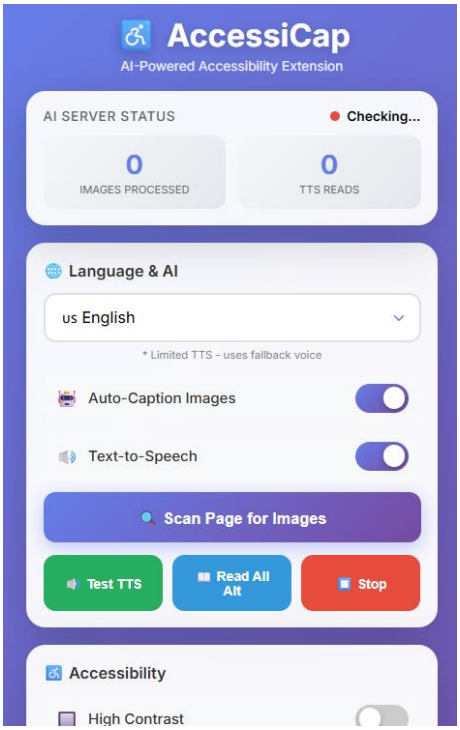
Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38–45. https://doi.org/10.18653/v1/2020.emnlp-demos.6

World Health Organization. (2020). World report on vision. WHO Press.

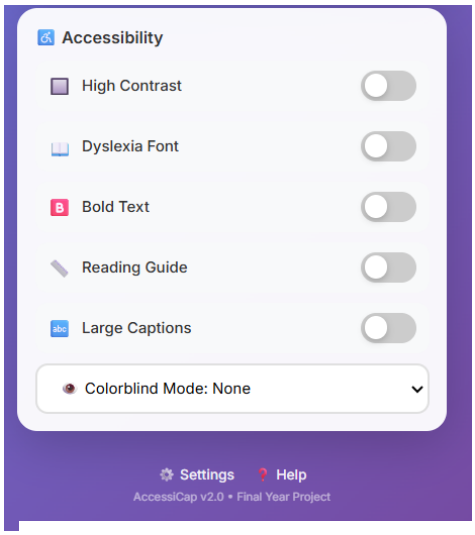
World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1 (W3C Recommendation). https://www.w3.org/TR/WCAG21/

Wu, H., Judd, P., Zhang, X., Isaev, M., & Micikevicius, P. (2020). Integer quantization for deep learning inference: Principles and empirical evaluation. arXiv preprint arXiv:2004.09602.

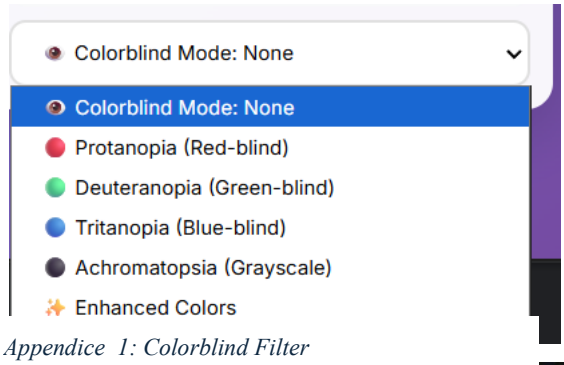
APPENDICES



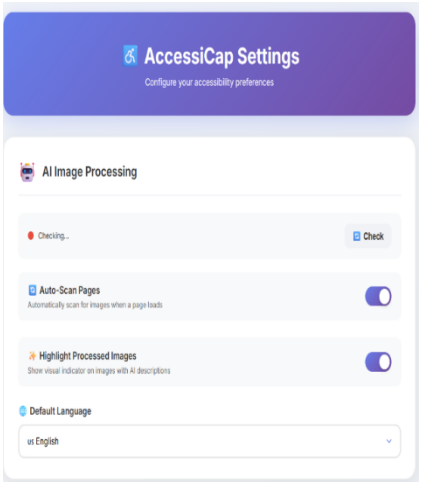
Appendice 1: Scan page & TTS



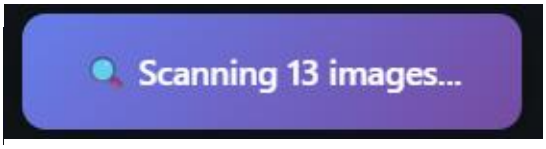
Appendice 2: Accessibility features



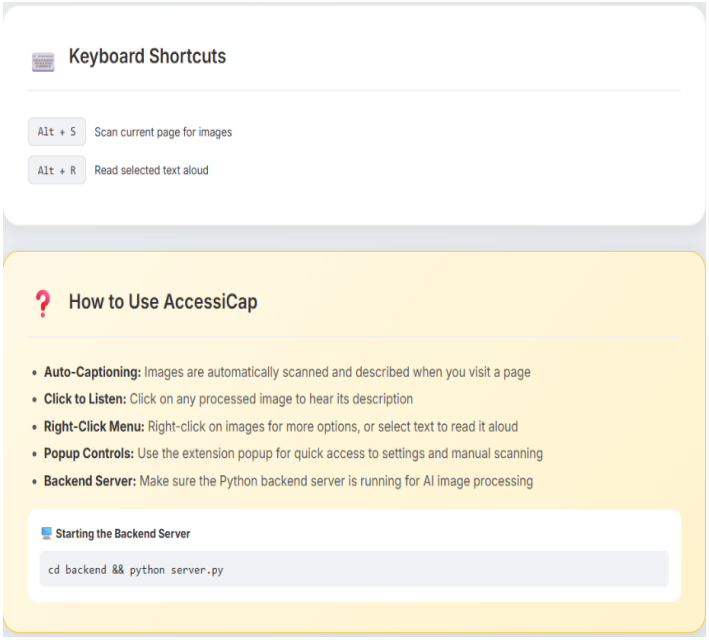
Appendice 1: Colorblind Filter



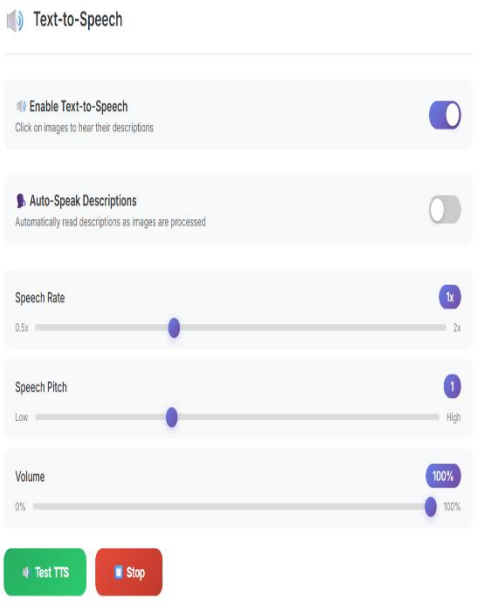
Appendice 2: Settings



Appendice 3: scanning pop up



Appendice 5: Keyboard Shortcut



Appendices 4: TTS settings

Appendices Code 1: FastAPI Endpoint (server.py)

```
@app.post("/caption", response_model=CaptionResponse)
async def generate_caption(request: ImageRequest):
    """
    Generate a caption with Celery and Redis cache.
    Returns immediately with a task_id if not cached.
    """
    try:
        image_data = request.imageData
        if "," in image_data:
            _, image_data = image_data.split(",", 1)
        img_bytes = base64.b64decode(image_data)
        image_hash = hashlib.sha256(img_bytes).hexdigest()
        cache_key = f"caption:{image_hash}:{request.language}"
        cached = await redis_client.get(cache_key)
        if cached:
            data = json.loads(cached)
            return CaptionResponse(
                caption=data["caption"],
                language=request.language,
                original_caption=data["caption"] if data["translated_text"] == data["caption"] else None,
                translated=data["translated_text"] != data["caption"]
            )
        # Run inference in background using Celery
        task = process_image_task.delay(img_bytes, request.language, cache_key=cache_key)
        return CaptionResponse(
            caption="Processing image...",
            language=request.language,
            task_id=task.id
        )
    except Exception as e:
        logger.error(f"Error processing image: {e}")
        return CaptionResponse(
            caption="Unable to process image at this time",
            language=request.language,
            error=str(e)
        )
```

```
@app.get("/result/{task_id}")
async def get_result(task_id: str):
    """Poll for the result of a caption generation task."""
    try:
        result = AsyncResult(task_id)
        if result.ready():
            if result.successful():
                res_data = result.get()
                return {"status": "completed", "result": res_data}
            else:
                return {"status": "error", "error": str(result.result)}
        return {"status": "pending"}
    except Exception as e:
        logger.error(f"Error polling task {task_id}: {e}")
        return {"status": "error", "error": str(e)}
```

Appendices Code 2: Celery Task (task.py)

```
def generate_caption(image_bytes: bytes) -> str:
    """Generate a caption using the ONNX encoder + PyTorch text decoder."""
    init_models()
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        if ort_session is not None:
            # Fast path: ONNX encoder → inject hidden states into text_decoder directly
            pixel_np = inputs["pixel_values"].numpy()
            encoder_hidden = ort_session.run(
                None, {"pixel_values": pixel_np}
            )[0]                                    # (1, seq, hidden)
            image_embeds = torch.from_numpy(encoder_hidden)
            # Replicate BLIP generation setup to bypass PyTorch vision_model entirely
            batch_size = image_embeds.shape[0]
            image_attention_mask = torch.ones(image_embeds.size()[:-1], dtype=torch.long,
device=image_embeds.device)
            # Start token
            input_ids = (
                torch.LongTensor([[blip_model.decoder_input_ids,
blip_model.config.text_config.eos_token_id]])
                .repeat(batch_size, 1)
```

```
            .to(image_embeds.device)
        )
        input_ids[:, 0] = blip_model.config.text_config.bos_token_id
        output_ids = blip_model.text_decoder.generate(
            input_ids=input_ids[:, :-1],
            eos_token_id=blip_model.config.text_config.sep_token_id,
            pad_token_id=blip_model.config.text_config.pad_token_id,
            encoder_hidden_states=image_embeds,
            encoder_attention_mask=image_attention_mask,
            max_new_tokens=50,
        )
    else:
        # Fallback: pure PyTorch
        output_ids = blip_model.generate(
            **inputs, max_new_tokens=50
        )
    caption = processor.decode(output_ids[0], skip_special_tokens=True)
    return caption
@app.task
def process_image_task(image_bytes: bytes, target_lang: str, cache_key: str = None):
    caption = generate_caption(image_bytes)
    audio_base64, translated_text = translate_and_speak(caption, target_lang)
    return {
        "caption": caption,
        "translated_text": translated_text,
        "audio_base64": audio_base64,
        "language": target_lang,}
```

Appendices Code 3: ONNX Inference (backend->tasks.py)

```
ort_session = None
blip_model   = None
processor    = None
translator   = Translator()
ONNX_MODEL_PATH = os.getenv("ONNX_MODEL_PATH", "blip_encoder_quantized.onnx")
def init_models():
    """Lazily load the ONNX encoder + PyTorch text decoder."""
    global ort_session, blip_model, processor
    if processor is not None:
```

```
        return
    model_name = "Salesforce/blip-image-captioning-base"
    print(f"[init] Loading processor from {model_name} ...")
    processor = BlipProcessor.from_pretrained(model_name)
    if os.path.exists(ONNX_MODEL_PATH):
        print(f"[init] Loading ONNX encoder from {ONNX_MODEL_PATH} ...")
        ort_session = ort.InferenceSession(
            ONNX_MODEL_PATH,
            providers=["CPUExecutionProvider"],
        )
        dummy = processor(
            images=Image.new("RGB", (384, 384)), return_tensors="np"
        )
        ort_session.run(None, {"pixel_values": dummy["pixel_values"]})
        print("[init] ONNX encoder warmed up.")
    else:
        print(
            f"[init] WARNING: {ONNX_MODEL_PATH} not found. "
            "Run convert_blip_to_onnx.py first. Falling back to full PyTorch model."
        )
    print(f"[init] Loading full BLIP model for text generation ...")
    blip_model = BlipForConditionalGeneration.from_pretrained(model_name)
    blip_model.eval()
    print("[init] Models ready.")
```

Appendices Code 4: Content Script (content.js)

```
async function processImage(img, speakResult = false) {
    img.classList.add('ac-processing');
    img.classList.remove('ac-processed', 'ac-error');
    if (settings.enableTts) {
      img.style.cursor = 'wait';
      img.onclick = (e) => {
        e.preventDefault();
        e.stopPropagation();
        speakText('Image is currently being processed. Please wait.');
      };
    }
    try {
```

```
      const imageData = await getImageAsBase64(img);
      if (!imageData) {
        throw new Error('Could not load image');
      }
      const response = await chrome.runtime.sendMessage({
        action: 'analyzeImage',
        imageData: imageData,
        language: settings.language
      });
      if (response && response.caption && response.error !== true) {
        // Handle pending task from Celery
        if (response.task_id) {
          pollForResult(response.task_id, img, speakResult);
          return;
        }
        applyCaptionToImage(img, response.caption, response.audio_base64, speakResult);
      } else {
        throw new Error('Server offline or image failed');
      }
    } catch (error) {
      console.log('AccessiCap: Skipping image -', error.message);
      img.classList.remove('ac-processing');
      if (img.onclick) {
        img.onclick = null;
        img.style.cursor = '';
      }
    }
  }
async function pollForResult(taskId, img, speakResult) {
    const check = async () => {
      try {
        const response = await chrome.runtime.sendMessage({
          action: 'pollResult',
          taskId: taskId
        });

        if (response && response.status === 'completed' && response.result) {
```

```
        applyCaptionToImage(img, response.result.caption, response.result.audio_base64,
speakResult);
       } else if (response && response.status === 'error') {
        console.error('AccessiCap: Task failed -', response.error);
        img.classList.remove('ac-processing');
        if (img.onclick) {
         img.onclick = null;
         img.style.cursor = '';
        }
       } else {
        setTimeout(check, 1000);
       }
      } catch(e) {
       setTimeout(check, 1000);
      }
    };
    check();
  }
```

Appendices Code 5: Background Service Worker (background.js)

```
function analyzeImage(imageData, language) {
  return new Promise((resolve, reject) => {
    fetch(`${SERVER_URL}/caption`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        imageData: imageData,
        language: language || 'en'
      })
    })
      .then(response => {
        if (!response.ok) {
          throw new Error('Server error: ' + response.status);
        }
        return response.json();
      })
      .then(data => {
        resolve(data);
```

```
    })
    .catch(error => {
      console.error('API Error:', error);
      resolve({
        caption: "Unable to analyze image - server may be offline",
        language: language || 'en',
        error: true
      });
    });
  });
}
function pollResultTask(taskId) {
  return new Promise((resolve, reject) => {
    fetch(`${SERVER_URL}/result/${taskId}`, { method: 'GET' })
      .then(response => {
        if (!response.ok) throw new Error('Server error');
        return response.json();
      })
      .then(data => resolve(data))
      .catch(error => {
        console.error('Poll error:', error);
        resolve({ status: 'error', error: true });
      });
  });
}
case "captureMagnifierTab":
    // Capture the visible tab screenshot for the magnifier lens
    chrome.tabs.captureVisibleTab(
      null,
      { format: 'jpeg', quality: 80 },
      (dataUrl) => {
        if (chrome.runtime.lastError || !dataUrl) return;
        // Push screenshot back to the content script in the sender tab
        if (sender && sender.tab && sender.tab.id) {
          chrome.tabs.sendMessage(sender.tab.id, {
            action: 'magnifierScreenshot',
            dataUrl: dataUrl
          }).catch(() => {});
```

```
        }
      }
    );
    sendResponse({ success: true });
    return true;
  }
});
function speakText(text, lang, rate, pitch, volume) {
  // Stop any ongoing speech
  chrome.tts.stop();

  chrome.storage.sync.get(['ttsRate', 'ttsPitch', 'ttsVolume', 'language'], (settings) => {
    const speechLang = lang || settings.language || 'en';
    const speechRate = rate || settings.ttsRate || 1.0;
    const speechPitch = pitch || settings.ttsPitch || 1.0;
    const speechVolume = volume || settings.ttsVolume || 1.0;
    console.log(`TTS: Speaking in lang='${speechLang}', text='${text.substring(0, 50)}...'`);
    chrome.tabs.query({ active: true, currentWindow: true }, (tabs) => {
      if (tabs[0]) {
        chrome.tabs.sendMessage(tabs[0].id, {
          action: 'webSpeakText',
          text: text,
          lang: speechLang,
          rate: speechRate,
          pitch: speechPitch,
          volume: speechVolume
        }).catch(() => {
          // Fallback to chrome.tts if content script not available
          chrome.tts.speak(text, {
            lang: speechLang,
            rate: speechRate,
            pitch: speechPitch,
            volume: speechVolume,
            onEvent: (e) => { if (e.type === 'error') console.error('TTS fallback error:',
e.errorMessage); }
          });
        });
      }
```

```
  });
 });
}
```

Appendices Code 6: Options Page (options.js)

```
function saveSettings() {
    const settings = {
      language: elements.language.value,
      autoCaption: elements.autoCaption.checked,
      highlightProcessed: elements.highlightProcessed.checked,
      enableTts: elements.enableTts.checked,
      autoSpeak: elements.autoSpeak.checked,
      highContrast: elements.highContrast.checked,
      dyslexiaFont: elements.dyslexiaFont.checked,
      readingGuide: elements.readingGuide.checked,
      boldText: elements.boldText.checked,
      largeCaptions: elements.largeCaptions.checked,
      colorblindMode: elements.colorblindMode.value,
      ttsRate: parseFloat(elements.ttsRate.value),
      ttsPitch: parseFloat(elements.ttsPitch.value),
      ttsVolume: parseFloat(elements.ttsVolume.value)
    };
    chrome.storage.sync.set(settings, () => {
      showToast('Settings saved successfully!', 'success');
      chrome.tabs.query({}, (tabs) => {
        tabs.forEach((tab) => {
          if (tab.url && !tab.url.startsWith('chrome://')) {
            chrome.tabs.sendMessage(tab.id, {
              action: 'settingsUpdated',
              settings: settings
            }).catch(() => { });
          }
        });
      });
    });
  }
async function checkServerStatus() {
    elements.serverStatusText.textContent = 'Checking...';
```

```
    try {
        const response = await chrome.runtime.sendMessage({ action: 'checkServerStatus' });

        if (response && response.online) {
            elements.serverStatusDot.classList.remove('offline');
            elements.serverStatusDot.classList.add('online');
            elements.serverStatusText.textContent = 'Server is online and ready!';
            elements.serverStatusText.style.color = '#27ae60';
        } else {
            setOffline();
        }
    } catch (error) {
        setOffline();
    }
}
```

Appendices Code 6: Docker File (backend->Dockerfile)

```
function saveSettings() {
    const settings = {
        language: elements.language.value,
        autoCaption: elements.autoCaption.checked,
        highlightProcessed: elements.highlightProcessed.checked,
        enableTts: elements.enableTts.checked,
        autoSpeak: elements.autoSpeak.checked,
        highContrast: elements.highContrast.checked,
        dyslexiaFont: elements.dyslexiaFont.checked,
        readingGuide: elements.readingGuide.checked,
        boldText: elements.boldText.checked,
        largeCaptions: elements.largeCaptions.checked,
        colorblindMode: elements.colorblindMode.value,
        ttsRate: parseFloat(elements.ttsRate.value),
        ttsPitch: parseFloat(elements.ttsPitch.value),
        ttsVolume: parseFloat(elements.ttsVolume.value)
    };
    chrome.storage.sync.set(settings, () => {
        showToast('Settings saved successfully!', 'success');
        chrome.tabs.query({}, (tabs) => {
```

```
            tabs.forEach((tab) => {
                if (tab.url && !tab.url.startsWith('chrome://')) {
                    chrome.tabs.sendMessage(tab.id, {
                        action: 'settingsUpdated',
                        settings: settings
                    }).catch(() => { });
                }
            });
        });
    });
}
async function checkServerStatus() {
    elements.serverStatusText.textContent = 'Checking...';
    try {
        const response = await chrome.runtime.sendMessage({ action: 'checkServerStatus' });

        if (response && response.online) {
            elements.serverStatusDot.classList.remove('offline');
            elements.serverStatusDot.classList.add('online');
            elements.serverStatusText.textContent = 'Server is online and ready!';
            elements.serverStatusText.style.color = '#27ae60';
        } else {
            setOffline();
        }} catch (error) {
        setOffline();}
}
```

Appendices Code 7: Python Dependencies (backend->requirement.txt)

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8080 \
    HOST=0.0.0.0 \
    TRANSFORMERS_CACHE=/app/model_cache \
    HF_HOME=/app/model_cache
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
ARG HF_TOKEN=""
RUN python3 - <<'EOF'
import os, sys
token = os.getenv("HF_TOKEN", "")
try:from transformers import BlipProcessor, BlipForConditionalGeneration
    kwargs = {"token": token} if token else {}
    print("Downloading BLIP model weights...")
    BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base", **kwargs)
            BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base",
**kwargs)
    print("Model download complete.")
except Exception as e:
    print(f"Warning: could not pre-download model: {e}", file=sys.stderr)
    sys.exit(0)
EOF
EXPOSE 8080
CMD ["python", "server.py"]
```

Appendices Code 8: Docker Compose (backend->docker-compose.yml)

```
version: '3.8'
services:
  redis:
    image: redis:alpine
    ports:
      - "6379:6379"
  api:build: .
    ports:
      - "8001:8001"
    environment:
      - REDIS_URL=redis://redis:6379/0
      - HOST=0.0.0.0
      - PORT=8001
    depends_on:
      - redis
    volumes:
      - .:/app
  worker:
```

```
  build: .
  command: celery -A tasks worker --loglevel=info
  environment:
    - REDIS_URL=redis://redis:6379/0
  depends_on:
    - redis
  volumes:
    - .:/app
```